DROIDRUSH 2026

TEAM: SIMPLETONS

TEAM ID

15

TEAM INFO

Member 1: **Ayush Jadaun** (20244048, ECE, ayush-jadaun, ayush_jadaun) — team leader

Member 2: **Arnav Raj** (20244036, ECE, arnavraj-7, arnav050148)

Member 3: **Abhay Agarwal** (20245003, Electrical Engineering, Abhay030405, abhayagarwal3751)

DESCRIPTION OF THE PROBLEM

Problem statement: **The Blackout Leyline** (PS 07). When a flood, earthquake or grid failure takes down cell towers and Wi-Fi, phones go silent even when people are only a few hundred metres apart. Yet almost every phone still carries a Bluetooth Low Energy (BLE) radio that needs no infrastructure at all. One BLE link reaches only about 10–30 m indoors, but a chain of phones can carry a message much further if each phone relays it to the next.

We propose **Leyline**, an Android app that will turn every phone into a relay in a delay-tolerant mesh network. A message from phone A will reach phone C through phone B, even though A and C are never in range of each other. If no path exists at that moment, the message will be stored and forwarded the moment a neighbour appears. Leyline will work in airplane mode with only Bluetooth switched on, SOS messages will always travel ahead of ordinary chat, and every SOS will carry the sender's GPS position (GPS needs no network), so rescuers will know not only that someone needs help but where.

App name: Leyline

PROCESS FLOW OF THE APP

1. **First launch:** the app will generate a random 32-bit node ID and store it. Android rotates BLE addresses every few minutes for privacy, so all routing will use this ID instead of the address.
2. **Permissions:** the app will request Bluetooth scan/advertise/connect on Android 12+, location on Android 11 and below (required there for BLE scanning), plus notifications. Each permission will be explained before it is requested.
3. **Join the mesh:** a foreground service will advertise a shared Leyline service UUID carrying the node ID, scan for other Leyline nodes, connect to nearby ones as a GATT client, and run its own GATT server so others can connect to it.
4. **Send:** the user will type a message (under 280 characters) or press SOS. The router will give it a unique message ID, a TTL (3–5 hops) and a priority, and will **commit it to the outbox before anything is sent**.

5. **Transmit:** the packet will be encoded in our binary format, split into chunks that fit the MTU negotiated for that connection (about 180 bytes), and written to every connected neighbour.
6. **Relay:** each receiving phone will reassemble the chunks and check the message ID against its recently-seen set. Duplicates will be dropped. Otherwise the phone will deliver the message (if addressed to it or to everyone), decrement the TTL and, if the TTL allows, forward it to every neighbour except the one it came from.
7. **Store and forward:** if no neighbour is connected, the message will wait in the outbox. When a phone reconnects, pending messages will be replayed to the new neighbour.
8. **Receipt and display:** the destination will send a delivery receipt back along the reverse path. Each message will show sender, time and hop telemetry (hop count and the chain of relays it passed through).

FEATURES ANNOUNCEMENT

The features we plan to implement:

Foundation (pass tier)

- **BLE discovery:** foreground advertising and scanning on a shared service UUID; GATT connections with chunked writes sized from the negotiated MTU.
- **Store-and-forward flood router:** message-ID deduplication, TTL 3–5, persistent outbox replayed on reconnect.
- **Multi-hop SOS:** 1-hop and 2-hop SOS messages (under 280 characters) showing sender, timestamp and hop path.
- **Disaster mode:** full operation in airplane mode with Bluetooth on.

Victory tier

- **Background relay** in a foreground service with a priority queue: SOS will always go before chat.
- **Long-message reassembly**, automatic reconnect with backoff, SOS beacon mode, and **message signing** so that a relay will not be able to forge who sent a message.
- **Wi-Fi Direct escalation** for large payloads, and a **4+ node broadcast-storm stress test**.

Our additions beyond the problem statement

- **Location in every SOS:** GPS works without any network, so each SOS will carry coordinates and accuracy. Receivers will see the distance and compass direction to the sender.
- **Delivery receipts:** acknowledgements will travel back along the reverse path, so the sender will see "SOS reached 4 people, nearest 2 hops away" instead of hoping.
- **Live mesh map:** a real-time graph of nearby nodes, link strength and the path each message took.
- **Battery-aware relaying:** phones low on battery will relay less and announce it; healthier phones will carry more load.
- **One-tap SOS anywhere:** a Quick Settings tile will send an SOS without opening the app.
- **Mesh health dashboard:** live delivery rate, latency, duplicates suppressed and bytes relayed — measured, not claimed.

- **Engineering tooling:** a JVM simulator that will run dozens of virtual nodes on the same routing code, a topology mode that will force multi-hop paths in a small room (always shown in the UI), and an in-app test log.

External libraries/APIs

We compared each choice on the template's criteria: active development, maintainer, licence, community size and whether it supports what a mesh needs out of the box. GitHub figures are as of October 2026.

Need	Chosen	Alternatives considered	Evidence
GATT client and server	**Nordic Android-BLE-Library** 2.11 + ble-ktx	Raw Android BLE APIs Kable (JUUL Labs) Nordic Kotlin-BLE-Library	Nordic: 2,423★, BSD-3-Clause, maintained by Nordic Semiconductor, last commit Oct 2026. Kable: 1,201★, Apache-2.0 — client role only. Kotlin-BLE-Library: 530★, v2.0 still beta.
Advertising and scanning	**Android platform APIs**	Nordic scanner-compat library	Built into Android since API 21; we will target API 26+, so no compatibility layer will be needed.
Local storage	**Room** 2.8 (SQLite)	Raw SQLite SQLDelight	Room: Google/AndroidX, Apache-2.0. SQLDelight: 6,897★, Apache-2.0 — built for multiplatform; Leyline will be Android-only.
Concurrency	**kotlinx.coroutines** 1.11 + Flow	RxJava Threads + callbacks	13,823★, Apache-2.0, JetBrains. The current Android standard.
UI	**Jetpack Compose + Material 3**	XML Views	Google, Apache-2.0, BOM 2026.09. Google's recommended UI toolkit.
Packet encoding	**Our own binary format**	Protocol Buffers JSON	No dependency; every byte will be documented in our packet specification.
Testing	**JUnit 6** + coroutines-test	—	Standard JVM testing, EPL-2.0.

Why these choices

- **Nordic over raw APIs:** Android allows only one GATT operation at a time per connection, and its BLE stack has phone-specific failures (status 133, stale service caches). Nordic's library queues operations, retries and negotiates MTU, and it supports **both** client and server roles. Rebuilding that would mean re-solving solved problems instead of building the mesh.
- **Not Kable:** a mesh node must accept incoming connections (server role); Kable only supports the client role.
- **Not Nordic's Kotlin rewrite:** still in beta (2.0.0-beta07); we will not demo on a beta transport.
- **Room over SQLDelight:** both are solid; Room gives compile-time-checked SQL, transactions and Flow-based UI updates with less setup for an Android-only app.
- **Our own packet format over Protobuf/JSON:** JSON is far too large for 180-byte chunks; Protobuf adds a dependency and hides the wire layout the problem statement asks us to specify.

- **Dependency wiring:** we will use a single hand-written container instead of Hilt/Koin — no generated code, no runtime surprises, and every object's lifetime visible in one place.

Every library and AI tool we use will be disclosed in the project README.

Core Engineering Architecture

This section describes the architecture we will implement.

Infrastructure Layout & Data Flow

Leyline will have **no server**: every phone will be an equal node running the same four layers — Compose UI, a foreground MeshService, the routing core (**mesh-core**, pure Kotlin with no Android code) and the Bluetooth layer (**mesh-ble**). Because mesh-core will have no Android dependency, the exact routing code that runs on the phone will also run in unit tests and in a JVM simulator with dozens of virtual nodes.

When a user sends an SOS, it will flow: UI → MeshService → router (assigns ID, TTL and priority) → outbox row **committed to Room first** → codec → chunker (splits by MTU) → GATT write to each neighbour. Receiving will run the reverse: chunks → reassembler → codec → router (dedup → deliver → forward). Caching will be the seen-set of message IDs; storage will be Room (outbox, message history, receipts).

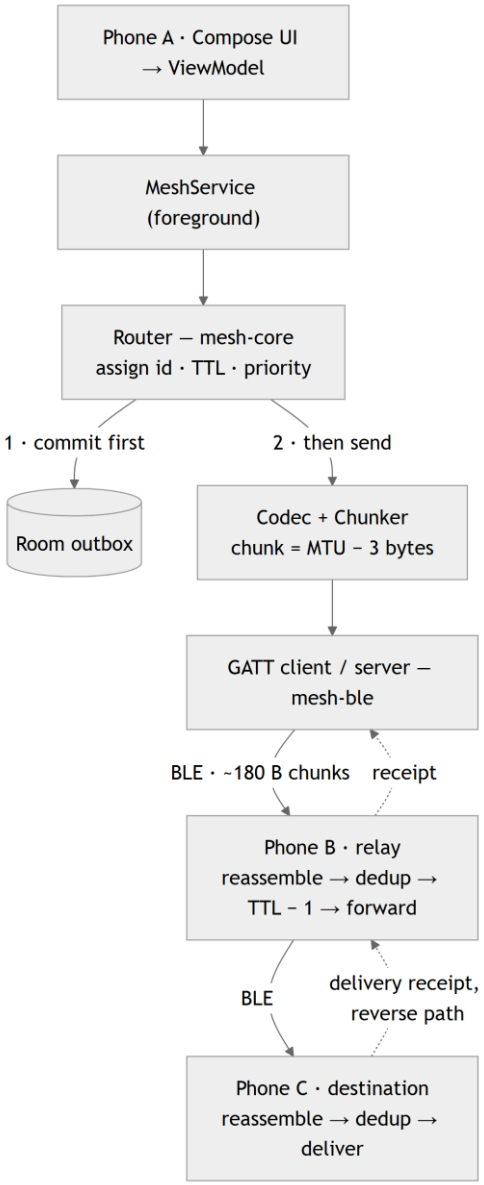


Figure 1 — Planned data flow for one SOS from phone A to phone C through relay B

Resource perspective: what happens under load

In a mesh, "heavy load" is not a server rush — it is a **broadcast storm**: twenty people in one shelter, every phone retransmitting every message. Unmanaged, the radios saturate and SOS messages drown. Every resource in Leyline will be bounded:

- **Radio:** each node will forward a message at most once (dedup). Rebroadcasts will wait a short random delay, and a node will skip forwarding if it has already heard enough neighbours forward the same message (counter-based suppression).
- **Connections:** the connection pool will keep a capped number of neighbours, preferring the strongest signal — protecting battery and staying within the per-phone connection limit.
- **Memory:** the seen-set will be bounded by size and by age, so a relay running for hours will not grow without limit.

- **Queue:** a bounded priority queue. SOS will never be dropped; when the queue is full, the oldest ordinary chat will be evicted first.
- **Battery:** scanning will run in duty cycles, and the service will run as a "connected device" foreground service so Android does not kill it mid-relay.

Production-grade, not a classroom assignment

Forwarding messages

- **The Naive/Basic Way:** Rebroadcast every packet received. Two phones in range bounce the same message back and forth forever, and in a crowd traffic multiplies at every hop until the links saturate.
- **Your Smart Engineering Way:** Every packet will carry a unique 64-bit message ID, a TTL and the list of relays it passed. A node will drop IDs it has seen, drop packets whose relay list already contains itself, decrement the TTL, never send back to the neighbour it came from, and suppress redundant rebroadcasts.
- **Why It Wins:** Each node will transmit each message at most once and no message will travel further than its TTL, so total traffic will be bounded and loops impossible. The README will include the proof, and every rule will be covered by unit tests.

Writing to BLE connections

- **The Naive/Basic Way:** Call a GATT write for each chunk the moment it is ready. Android allows only one GATT operation in flight per connection — the second call fails silently and chunks vanish.
- **Your Smart Engineering Way:** A per-connection serial operation queue with retry and exponential backoff on Android's common GATT failure codes (8 = connection timeout, 22 = terminated by local host, 133 = generic error).
- **Why It Wins:** Every chunk will be written in order or retried; an unstable link will cost latency, never corrupt a message.

Not losing messages

- **The Naive/Basic Way:** Keep pending messages in an in-memory list. If Android kills the app or the phone restarts, every undelivered SOS is gone.
- **Your Smart Engineering Way:** Write-ahead persistence: a message will be committed to the Room outbox before its first transmission. The message ID will be the primary key, so retries and duplicates cannot insert twice (idempotent). Delivery state will be tracked per neighbour and replayed on reconnect.
- **Why It Wins:** A message will survive process death, reboot and long disconnection; store-and-forward will resume exactly where it stopped.

Identifying phones

- **The Naive/Basic Way:** Use the Bluetooth MAC address as identity. Android rotates BLE addresses every few minutes for privacy, so the same phone shows up under several addresses within half an hour — dedup and routing break.
- **Your Smart Engineering Way:** A persistent random node ID carried inside every advertisement and every packet.
- **Why It Wins:** Identity will be independent of the radio layer, stay stable across rotations, and make message signing possible.

Packet size

- **The Naive/Basic Way:** Hard-code one size, or the 20-byte default payload. Android 14+ always negotiates MTU 517 while older phones settle on lower values such as 247 — a fixed size is either wasteful or broken.
- **Your Smart Engineering Way:** Chunk size will come from the MTU actually negotiated on each connection (MTU − 3 bytes of ATT header), capped at about 180-byte chunks that fit old and new phones alike.
- **Why It Wins:** It will work across old and new phones with no per-device configuration.

Hardware plan

Leyline will target Android 8.0+ (API 26), which provides both BLE roles a mesh node needs: GATT server and GATT client. The multi-device demo and the 4+ node stress test will use Android phones of different ages, so both MTU behaviours (517 on Android 14+, lower values on older versions) will be covered.

CLASS DIAGRAM

mesh-core (Packet, PacketCodec, SeenCache, Router, ForwardQueue, Chunker, Reassembler) will be pure Kotlin. **mesh-ble** will implement the Transport interface over GATT; the **app** module will implement Outbox with Room and host MeshService and the UI. Interfaces at the module boundaries will let the simulator replace BLE and Room with in-memory fakes.

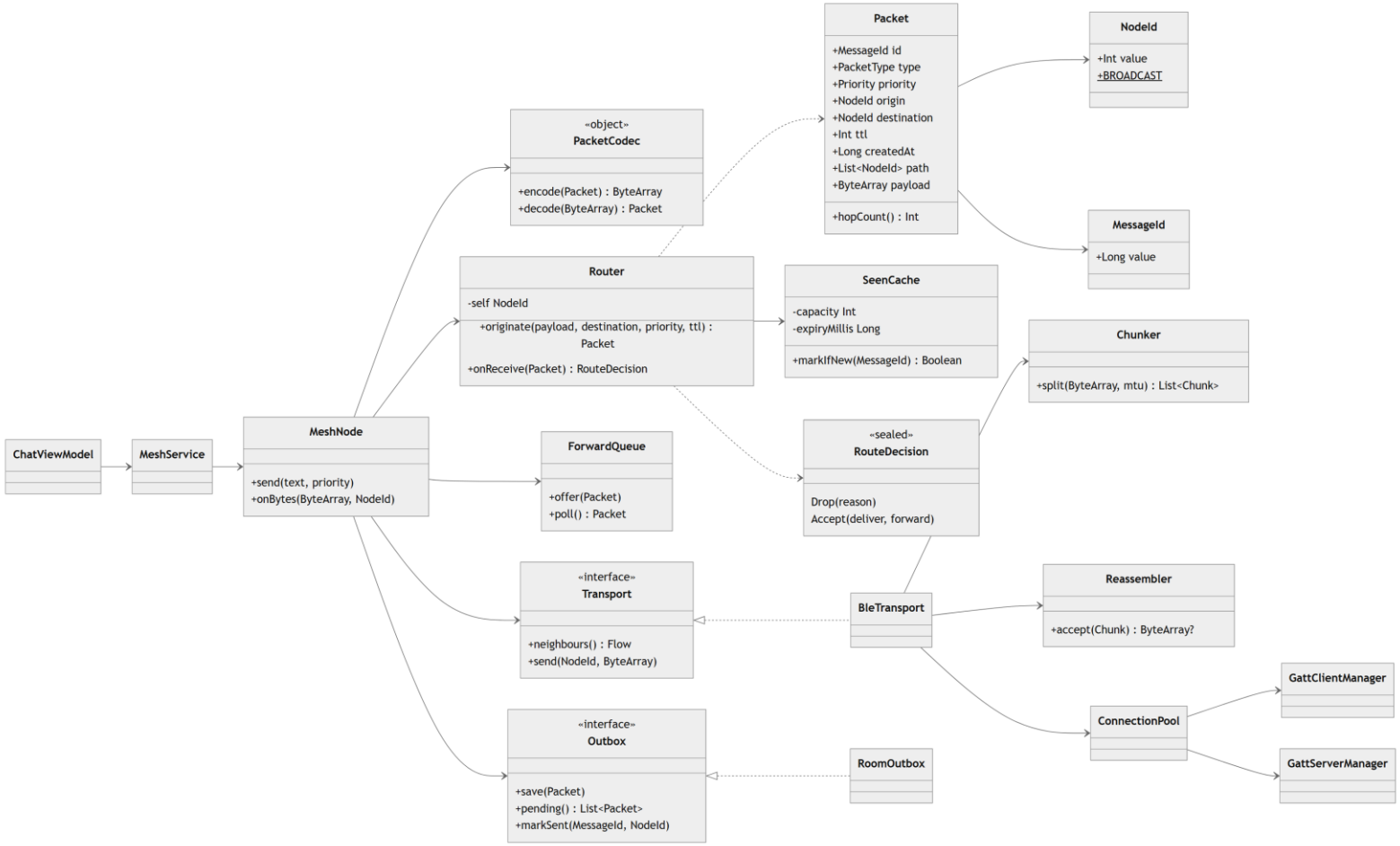


Figure 2 — Planned class diagram of Leyline